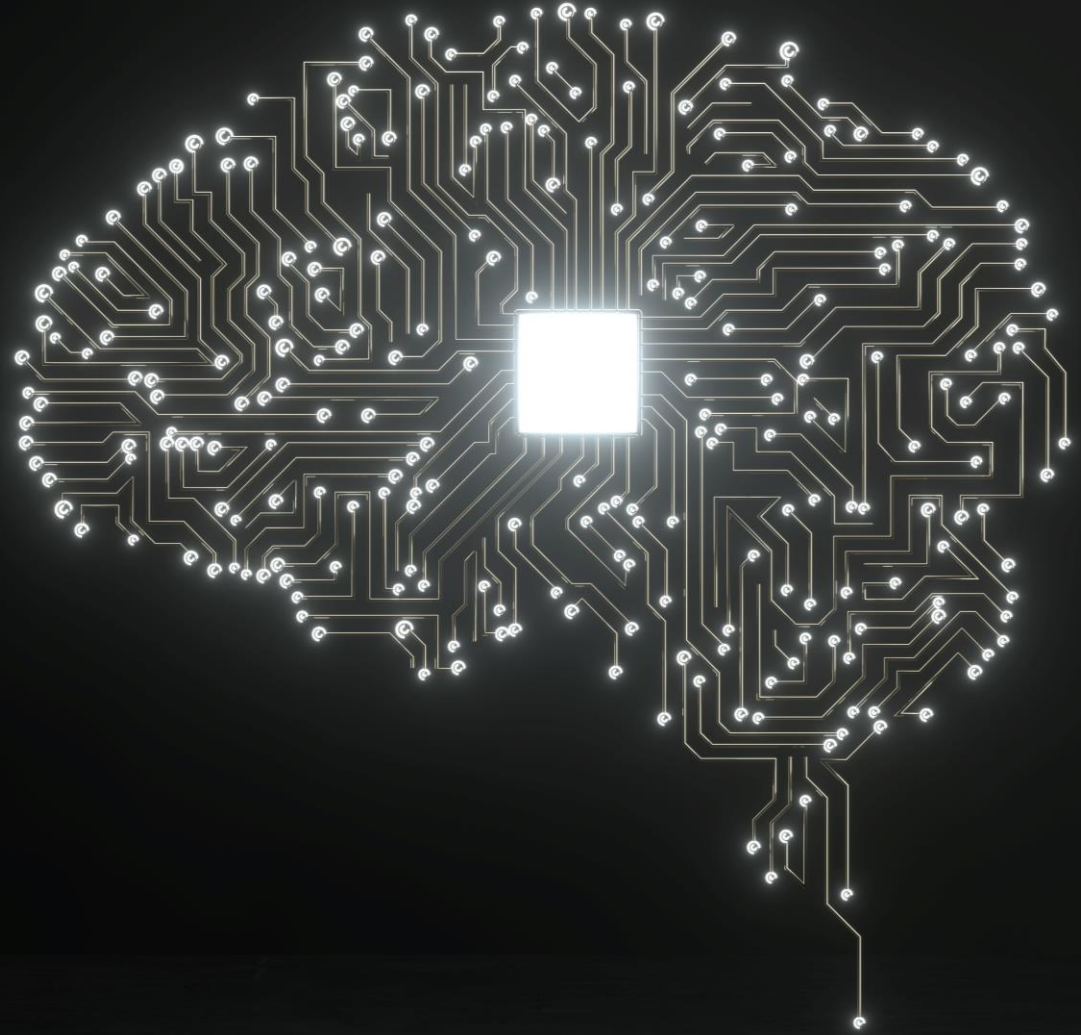


Infrastructure as Code – Configuration Management Part 1

WELCOME TO CLASS 5!



Goals of the day



The origins of Configuration Management



How does Ansible work?



Hands on Ansible



Photo by [David Becker](#) on [Unsplash](#)

Agenda

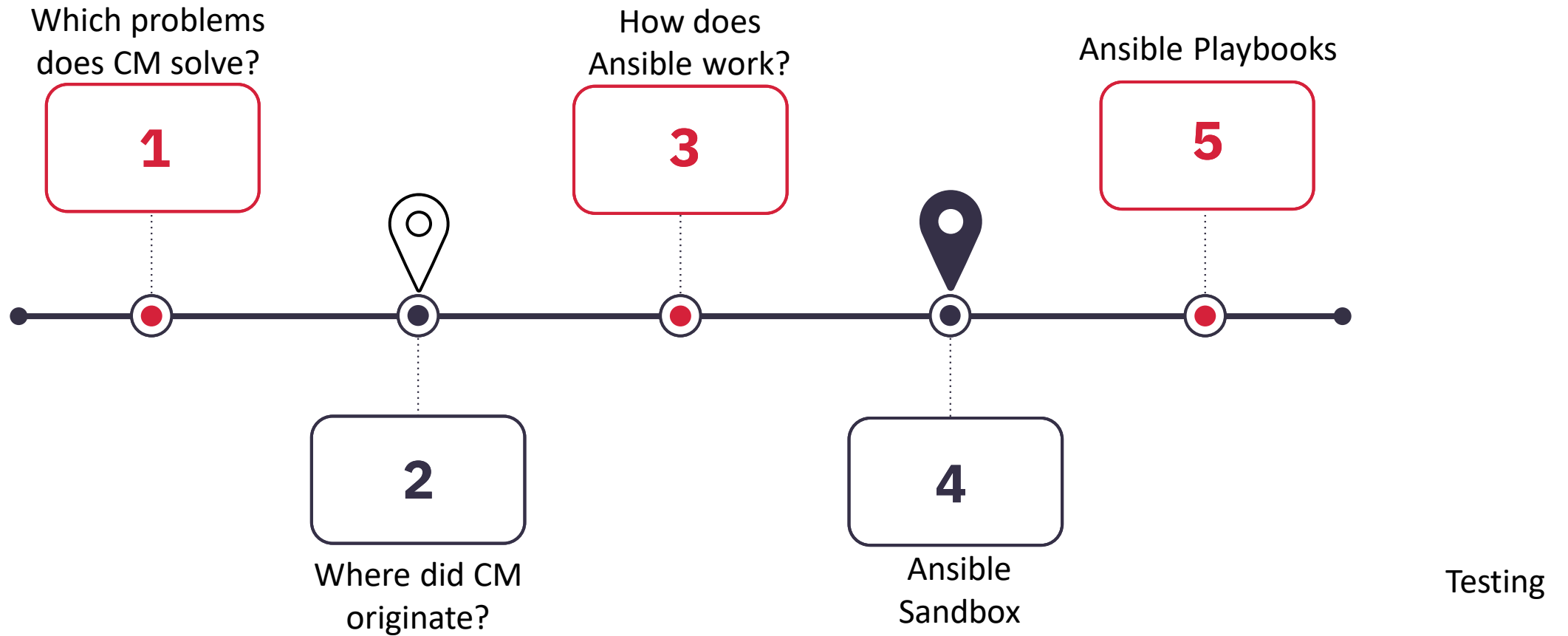
- Origins of Configurations Management (CM) – what problems does it solve and where did it come from?
- **Break**
- How does Ansible work?
- Creating an Ansible sandbox
- Our first Ansible playbook

Photo by David Becker on Unsplash

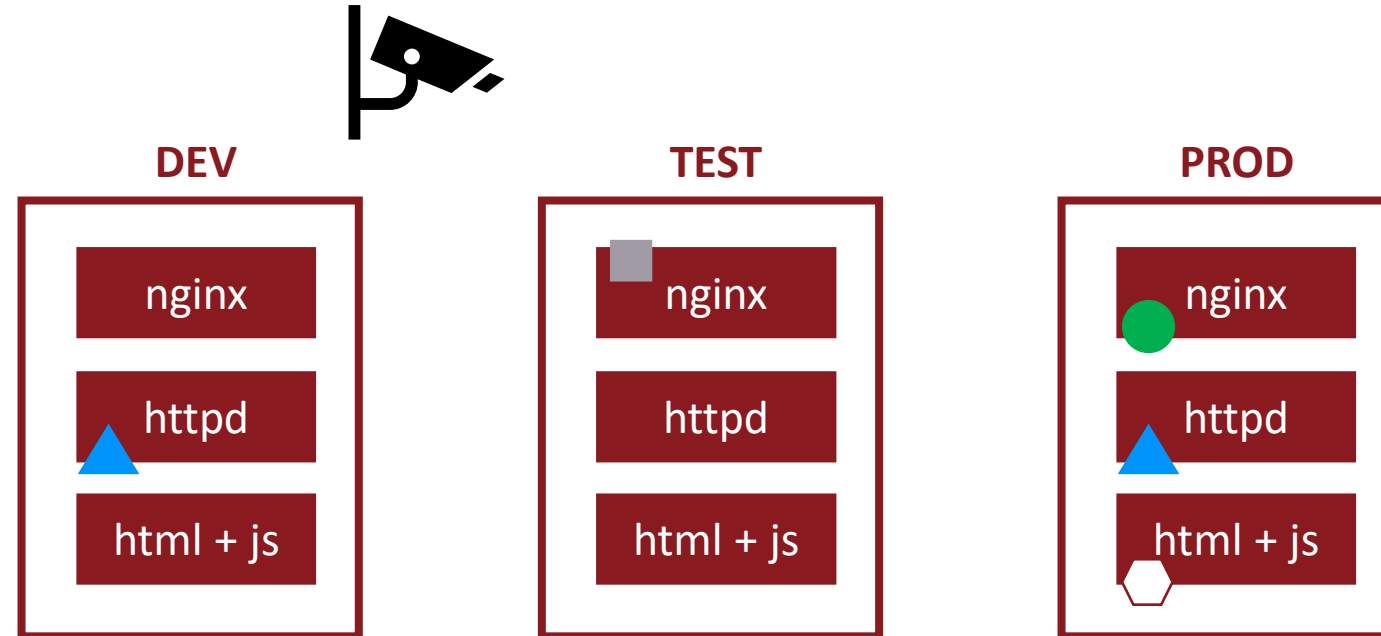
Review

- How did the exercises from last class go for you?
- How confident do you feel that Terraform is a suitable tool for managing infrastructure provisioning?
- How confident do you feel in your ability to understand and develop Terraform code?
- What would help you get to a higher level with Terraform?
- What is your level of knowledge like with Ansible?
- Brief Q & A

Journey Map



Why we need to manage configuration?



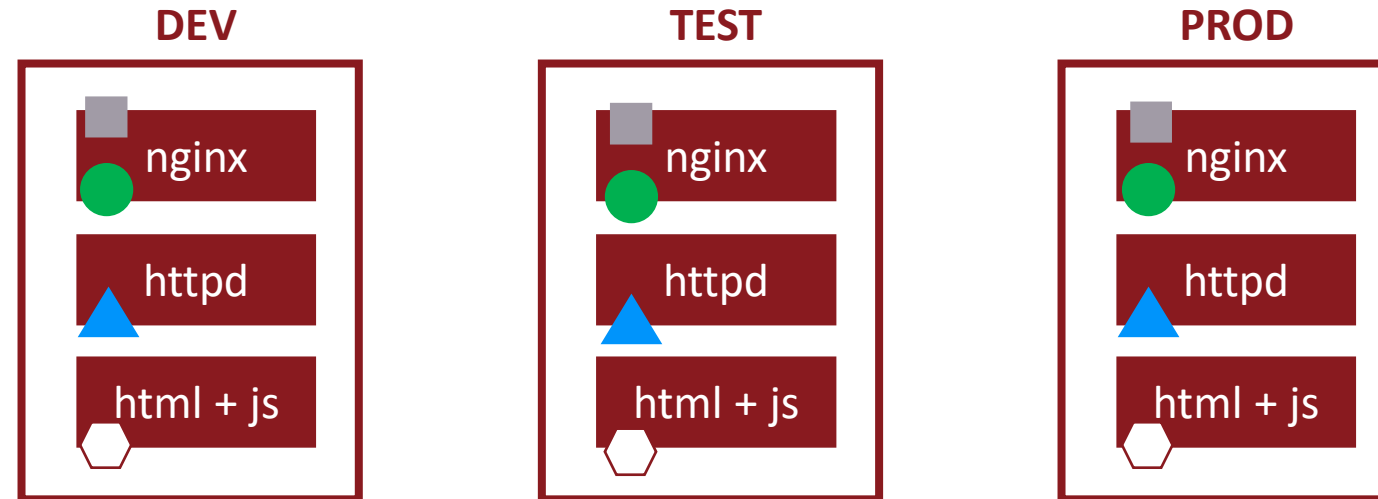
When configuration changes are made manually, they can be inconsistent and error prone

Change can be more difficult to govern, particularly in response to incidents.

Security scanning is not always applied equally across environments.

This becomes problematic because the code is developed and signed off in environments that are different to production, which can cause incidents in production.

Why we need to manage configuration? (con't)



Instead of this ad hoc process, the DevOps practice of Configuration Management ensures there is consistency across environments.

Changes are made in sequence through environments and verified as we progress.

There provides a clear audit trail and a known state for each element in the estate.

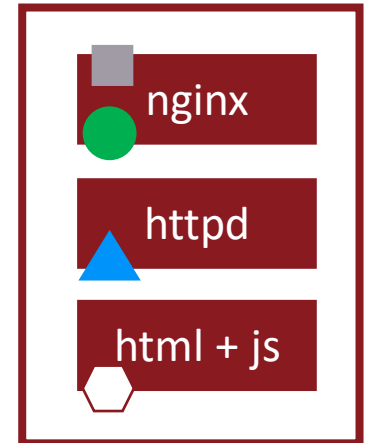
It also creates more predictable outcomes in software delivery, which is critical.

What is Configuration Management?

Configuration Management (CM) is used to document, maintain and apply change management to configuration updates in infrastructure, using automation and at scale.*

When CM is done well, the following benefits are gained: **

- Improved visibility and tracking of changes, reducing outages and security breaches
- Reduced costs due to clarity on state of all technology resources
- Greater platform stability, which improves customer and staff experience
- Greater control of changes applied to systems, with greatly improved oversight
- Faster mean time to recovery in the event of issues
- Clarity in change and release management functions as to infrastructure changes.



* <https://www.redhat.com/en/topics/automation/what-is-configuration-management>

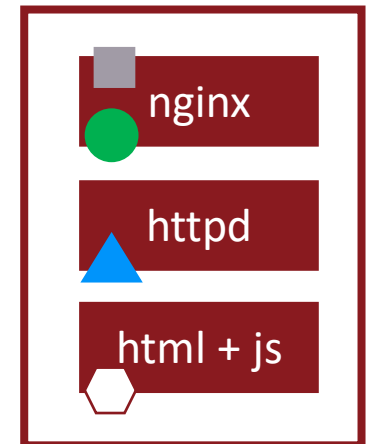
** <https://www.upguard.com/blog/5-configuration-management-boss>

What is Configuration Management? (con't)

Configuration Management (CM) is used to document, maintain and apply change management to configuration updates in infrastructure, using automation and at scale.*

When CM is not done well, the following issues arise: **

- A lot of manual work to assess what needs to change with new requirements
- Failure due to lack of clear communication to all stakeholders
- Time lost in slow rollbacks of new versions when issues arise
- Unexpected and unexplained outages due to decisions made based on incomplete information.



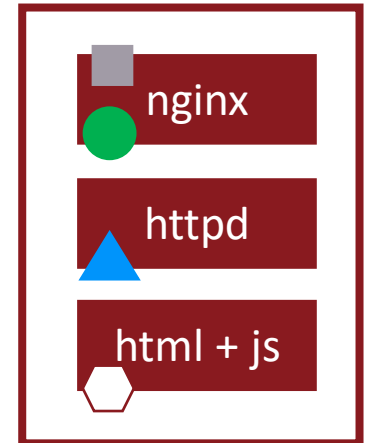
* <https://www.redhat.com/en/topics/automation/what-is-configuration-management>

** <https://www.upguard.com/blog/5-configuration-management-boss>

Origins of Configuration Management

In the example showing why we need CM, we should question how the patching, updates and upgrades were carried out before tools like Ansible were created.

- Typically, an admin would ssh to a server, with credentials maintained in a keystore at best, or more likely in a text file on their desktop. Once on the server, they would run commands using sudo permissions to update packages and configuration. There may be slight differences between one server and the next, which the admin would deal depending on their skill level. Highly subjective.
- Many admins started to create shell scripts to managed these activities, a case of working hard to be lazy. These scripts are brittle, as if there are differences from one environment to the next, they will fail. You then need to unravel everything that happened to the failure point, fix the issue and re-run. Problematic.
- Along comes Ansible (and other tools) and CM is born!



* <https://www.redhat.com/en/topics/automation/what-is-configuration-management>

** <https://www.upguard.com/blog/5-configuration-management-boss>

Recap on IaC Options [from course 4, class 2]

	Terraform	Ansible	Cloud Formation	Chef	Puppet
Type	Provisioning	Config Mgmt	Provisioning	Config Mgmt	Config Mgmt
Infra	Immutable	Mutable	Immutable	Mutable	Mutable
Paradigm	Declarative	Imperative	Declarative	Imperative	Imperative
Source	Open	Open	Closed	Open	Open
Community	Huge	Huge	AWS	Large	Large
Cloud	All	All	Small	All	All

Adapted from Terraform Up & Running, 3rd Edition, Brikman, Yevgeniy, O'Reilly, 2022

Since we reviewed this table, we've learned about the extensive provisioning capabilities of Terraform.

Turning to Ansible now, we can see it is the clear leader in the Configuration Management space.

We will spend the remainder of this class and all of the next looking at how Ansible works, how to set it up and reviewing several examples for which you will get source code.

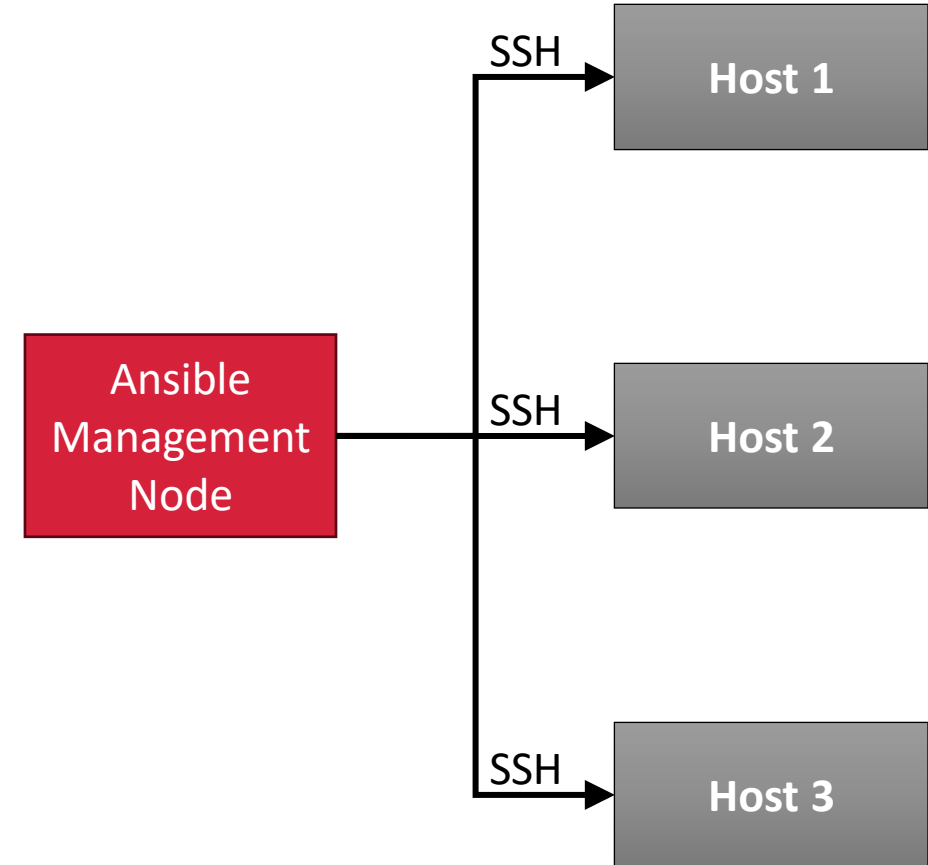
Break (15 Minutes)



How does Ansible Work? *

Modules: Ansible connects to target hosts, using library plugins called modules that can be used in CLI Ansible or as code to push commands to these target hosts over SSH. Modules encapsulate complexity and help achieve idempotency in Ansible.

Agentless: One of the big selling points of Ansible is that you don't need to install agents. SSH keys enable connectivity, Ansible can sudo to root if required once connected. Modules are transferred to hosts for execution.



* <https://www.redhat.com/en/topics/automation/learning-ansible-tutorial>

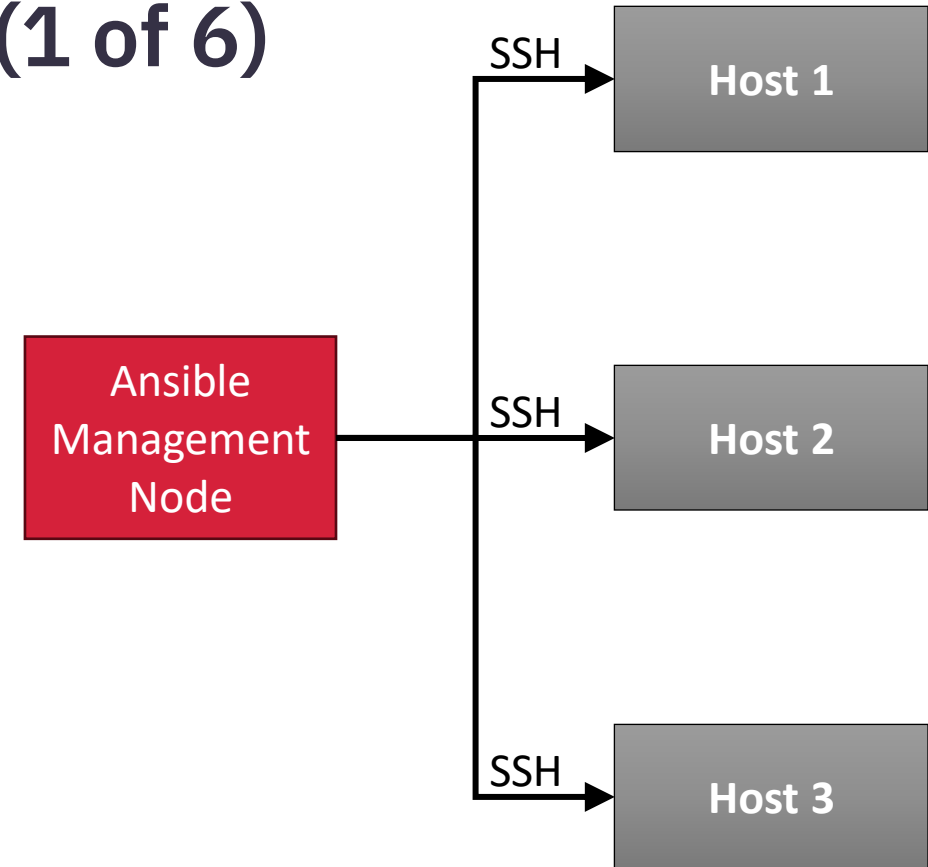
Exercise 1 – Ansible Sandbox

Let's start with something simple (1 of 6)

Although this diagram shows a management (or control) node interacting with several remote hosts, the simplest scenario is Ansible applying updates on the localhost.

The term localhost refers to the computer you are working on. Right now, that's your laptop. If you create an EC2 and SSH to it, then localhost is that EC2. Also denoted as 127.0.0.1, which is the private IP address of the local computer. *

If you run a webserver on a machine, then it can be accessed using <http://localhost:80> or <http://127.0.0.1> locally, but not from any other machine.



* <https://www.baeldung.com/cs/127-0-0-1-vs-localhost>

Let's start with something simple (2 of 6)

We will progress to running Ansible remotely, but first let's run it locally. To get started, we will create an EC2 with Ansible installed, then SSH to it and try some Ansible commands.

1. Clone the Terraform code from GitHub repo.
2. Create the EC2 and SSH to the server.
3. Type `ansible --version` to confirm that Ansible was installed successfully.
4. You are now ready to try out some Ansible commands to better understand this technology.

```
Changes to Outputs:
+ Ansible-Sandbox-PublicIP = (known after apply)
aws_key_pair.aws-key: Creating...
aws_security_group.instance: Creating...
aws_key_pair.aws-key: Creation complete after 0s [id=ansible]
aws_security_group.instance: Creation complete after 2s [id=sg-041f14f1590231e17]
aws_instance.example: Creating...
aws_instance.example: Still creating... [10s elapsed]
aws_instance.example: Still creating... [20s elapsed]
aws_instance.example: Still creating... [30s elapsed]
aws_instance.example: Still creating... [40s elapsed]
aws_instance.example: Creation complete after 42s [id=i-046810699696196f8]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:
Ansible-Sandbox-PublicIP = "34.230.89.233"
```

```
[shane@Shanes-MacBook-Pro ~ % ssh ec2-user@34.230.89.233
The authenticity of host '34.230.89.233 (34.230.89.233)' can't be established.
ED25519 key fingerprint is SHA256:FFsPMa07RM0eGjhBDJms6oQqmsrzPYURw5s9cUjDxNs.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '34.230.89.233' (ED25519) to the list of known hosts.
```

```

  ____  _
 _||_  (  /  Amazon Linux 2 AMI
---| \___|___|
```

```
https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-172-31-24-136 ~]$ ansible --version
ansible 2.9.23
  config file = /etc/ansible/ansible.cfg
  configured module search path = [u'/home/ec2-user/.ansible/plugins/modules', u'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python2.7/site-packages/ansible
  executable location = /usr/bin/ansible
  python version = 2.7.18 (default, Feb 28 2023, 02:51:06) [GCC 7.3.1 20180712 (Red Hat 7.3.1-15)]
```

Let's start with something simple (3 of 6)

First, we can start with running ad hoc Ansible commands, which have the following format:

```
ansible [pattern] -m [module] -a "[module options]"
```

Where:

pattern identifies the servers on which to run the command

module is one of the library plugins we mentioned earlier

options are arguments passed to the module as part of execution

For example:

```
ansible localhost -m shell -a "free -m"
```

```
[ec2-user@ip-172-31-24-136 ~]$ ansible localhost -m shell -a "free -m"
localhost | CHANGED | rc=0 >>
      total        used          free      shared  buff/cache   available
Mem:      970          174          171           0          624          644
Swap:      0            0            0
```

Let's start with something simple (4 of 6)

Then, we can progress to ad hoc commands that do more. We will use Ansible to install and run Apache httpd.

```
[[ec2-user@ip-172-31-24-136 ~]$ ansible localhost -b -m yum -a "name=httpd state=present"
localhost | CHANGED => {
  "ansible_facts": {
    "pkg_mgr": "yum"
  },
  "changed": true,
  "changes": {
    "installed": [
      "httpd"
    ]
  },
}
```

The `pattern` is `localhost`, so that the change is only applied on the machine on which the command is run.

We use the `yum` module to install `httpd`.

This requires the `-b` switch, which indicates becomes, executing the command as though with `sudo`.

The output shows that `localhost` was changed and `httpd` was installed.

Let's start with something simple (5 of 6)

Check the status of the install before proceeding.

```
[[ec2-user@ip-172-31-24-136 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled;
  Active: inactive (dead)
```

You can see it is disabled. This can be resolved with the next command, which starts and enables the httpd service.

```
[[ec2-user@ip-172-31-24-136 ~]$ ansible localhost -b -m systemd -a "name=httpd state=started enabled=true"
localhost | CHANGED => {
  "changed": true,
  "enabled": true,
  "name": "httpd",
  "state": "started",
```

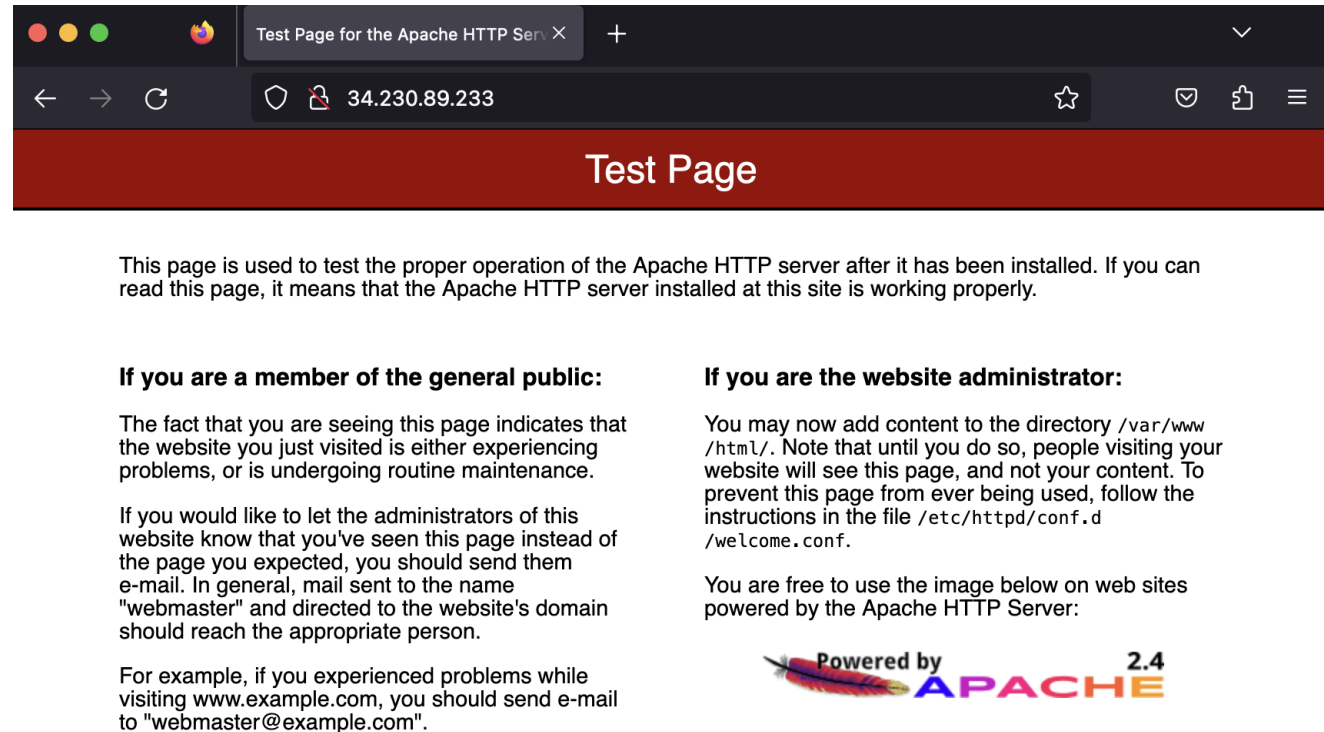
Finally, check the status again to verify all is well.

```
[[ec2-user@ip-172-31-24-136 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled;
  Active: active (running) since Fri 2023-07-07 02:12:09 UTC; 40m
  Docs: man:httpd.service(8)
```

Let's start with something simple (6 of 6)

At this point, you can also verify that the httpd service is running from your laptop, by entering the IP address of this server in your browser.

The default page is shown.



Next, we will bundle this series of commands into an Ansible file, called a playbook and execute it as part of creating the Ansible EC2.

Exercise 2 – Ansible Webserver Playbook

Ansible Playbook

The ad hoc Ansible commands we used to install and run a webserver can be packaged as code in the form of an Ansible Playbook.

This code is in YAML format, which provides a readable, concise way of defining steps in your configuration management.

Our first playbook will use the modules from the ad hoc commands earlier, with some minor changes to the terraform that created the EC2.



Photo by [Mario Verduzco](#) on [Unsplash](#)

Ansible Playbook – Webserver (1 of 5)

```
resource "aws_instance" "example" {
  ami           = data.aws_ssm_parameter.webserver-ami.value
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name      = aws_key_pair.aws-key.key_name

  user_data = <<-EOF
  ...
  sudo yum update -y && sudo amazon-linux-extras install ansible2 -y
  ...
  EOF

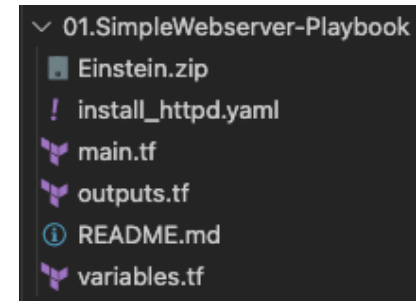
  tags = {
    Name = "ansible-sandbox"
  }
}
```

Ansible Sandbox

First, we'll review the differences between the Ansible Sandbox terraform and the Ansible Playbook terraform.

- Addition of file provisioners to copy files to EC2.
- Removal of user_data and addition of remote_exec provisioner to install dependencies and execute playbook.

Next, let's review the install_httpd.yml playbook.



Folder Structure

```
resource "aws_instance" "example" {
  ami           = data.aws_ssm_parameter.webserver-ami.value
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name      = aws_key_pair.aws-key.key_name

  tags = {
    Name = "ansible_install_httpd"
  }

  connection {
    type = "ssh"
    user = "ec2-user"
    private_key = file(var.ssh_key_private)
    host = self.public_ip
  }

  # Copy the playbook from local machine to EC2
  provisioner "file" {
    source = "${path.root}/install_httpd.yml"
    destination = "install_httpd.yml"
  }

  # Copy the webserver content from local machine to EC2
  provisioner "file" {
    source = "${path.root}/Einstein.zip"
    destination = "Einstein.zip"
  }

  # Execute a script on a remote resource
  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y && sudo amazon-linux-extras install ansible2 -y",
      "sleep 60s",
      "ansible-playbook install_httpd.yml"
    ]
  }
}
```

Ansible Playbook

Ansible Playbook – Webserver (2 of 5)

Three dashes at the start indicate a yml file.

- **hosts** specifies on which servers to execute the playbook.
We'll discuss this in more detail when we get to inventories.
- **remote_user** is who is running the playbook.
- **become** when set to true is equivalent to sudo command.

Next, a list of tasks to be executed are specified. You will likely recognize the modules from the ad hoc commands run on the server earlier.

```
- name: Install httpd
  hosts: localhost
  remote_user: ec2-user
  become: true
  tasks:
    - name: install yum httpd package
      yum:
        name: httpd
        state: present
    - name: Unarchive a file that is already on the remote machine
      unarchive:
        src: /home/ec2-user/Einstein.zip
        dest: /tmp
        remote_src: yes
    - name: Copy files
      copy:
        src: '/tmp/{{item}}'
        dest: /var/www/html/
        owner: ec2-user
      loop:
        - index.html
        - einstein.jpg
    - name: Start and Enable httpd
      systemd:
        name: httpd
        state: started
        enabled: true
```

Ansible Playbook

Ansible Playbook – Webserver (3 of 5)

- **yum** command syntax is equivalent to this ad hoc command

```
[ec2-user@ip-172-31-24-136 ~]$ ansible localhost -b -m yum -a "name=httpd state=present"
localhost | CHANGED => {
  "ansible_facts": {
    "pkg_mgr": "yum"
  },
  "changed": true,
  "changes": {
    "installed": [
      "httpd"
    ]
  }
}
```

- **unarchive** decompresses the zip file copied over by the file provisioner and places content in a /tmp folder
- **copy** uses a loop and keyword **item** to copy the contents of that archive into the root folder of the httpd service
- Finally, the **systemd** module ensures httpd is enabled and started, as per earlier ad hoc command.

```
[ec2-user@ip-172-31-24-136 ~]$ ansible localhost -b -m systemd -a "name=httpd state=started enabled=true"
```

```
- name: Install httpd
  hosts: localhost
  remote_user: ec2-user
  become: true
  tasks:
    - name: install yum httpd package
      yum:
        name: httpd
        state: present

    - name: Unarchive a file that is already on the remote machine
      unarchive:
        src: /home/ec2-user/Einstein.zip
        dest: /tmp
        remote_src: yes

    - name: Copy files
      copy:
        src: '/tmp/{{item}}'
        dest: /var/www/html/
        owner: ec2-user
      loop:
        - index.html
        - einstein.jpg

    - name: Start and Enable httpd
      systemd:
        name: httpd
        state: started
        enabled: true
```

Ansible Playbook

Ansible Playbook – Webserver (4 of 5)

Reviewing ansible output from terraform apply, you can see each of the tasks from the previous slide with a status on what happened and if each step was OK.

Next, check if the webpage archive that we copied to the EC2 was rendered successfully by viewing **Ansible-Install_httpd-PublicIP** in a browser.

```
aws_instance.example (remote-exec): PLAY [Install httpd] *****
aws_instance.example (remote-exec): TASK [Gathering Facts] *****
aws_instance.example (remote-exec): ok: [localhost]

aws_instance.example (remote-exec): TASK [install yum httpd package] *****
aws_instance.example (remote-exec): Still creating... [2m10s elapsed]
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [Unarchive a file that is already on the remote machine] *****
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [Copy files] *****
aws_instance.example (remote-exec): changed: [localhost] => (item=index.html)
aws_instance.example (remote-exec): changed: [localhost] => (item=einstein.jpg)

aws_instance.example (remote-exec): TASK [Start and Enable httpd] *****
aws_instance.example (remote-exec): Still creating... [2m20s elapsed]
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): PLAY RECAP *****
aws_instance.example (remote-exec): localhost : ok=5 changed=4 unreachable=0 failed=0 skipped=0

aws_instance.example: Creation complete after 2m21s [id=i-0196feb83009c1bf1]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:
Ansible-Install_httpd-PublicIP = "35.175.217.81"
```

Photo by [Mario Verduzco](#) on [Unsplash](#)

Date, Time, and Server Name

Current Date and Time: 2023-10-22, 10:20:13 a.m.

Server Name: 3.80.215.71

Top 10 Albert Einstein Quotes

"Imagination is more important than knowledge."

"The only source of knowledge is experience."

"Strive not to be a success, but rather to be of value."

"The true sign of intelligence is not knowledge but imagination."

"Logic will get you from A to B. Imagination will take you everywhere."

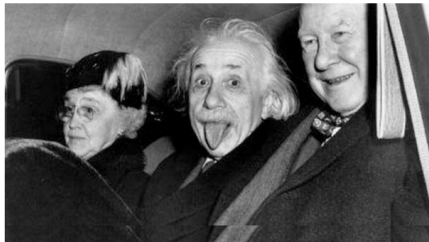
"The difference between stupidity and genius is that genius has its limits."

"Education is what remains after one has forgotten what one has learned in school."

"In the middle of difficulty lies opportunity."

"Look deep into nature, and then you will understand everything better."

"Imagination is everything. It is the preview of life's coming attractions."



Ansible Playbook – Webserver (5 of 5)

It all looks good!

You'll notice that we didn't SSH to this server to complete any of this work, it was all completed as part of the provisioning workflow, with the configuration management layered on. You even verify the outcome from your laptop, without a local verification on the EC2.

You can log on to the server now and verify the httpd process is running and make updates to the index.html file if you wish.

Exercise 3 – Ansible Maria DB Playbook

Ansible Playbook – Database (1 of 6)

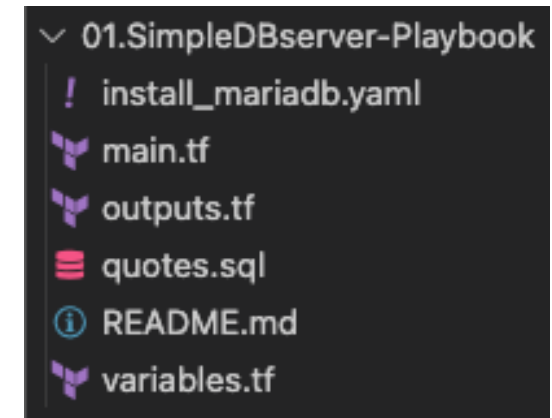
- Now that we understand how to create a playbook for a webserver, let's look at what is involved in creating a playbook for a database server.
- For this example, we will use MariaDB which is an easy-to-use MySQL implementation. Through an Ansible playbook we will install, enable and start the database, then create a new schema in that database, all through Ansible.
- First, we will review the terraform, then the playbook.



Photo by [Mario Verduzco](#) on [Unsplash](#)

Ansible Playbook – Database (2 of 6)

- Here's an overview of the files in this project.
- It is similar to the webserver code, with a sql file added instead of the zip file, which contained the html and jpg files.
- Next, we'll review the terraform code, to highlight changes since the webserver implementation.



Ansible Playbook – Database (3 of 6)

As you can see, this is very similar to the webserver terraform code.

Key differences include:

1. install_mariadb.yaml ansible file is copied to EC2.
2. quotes.sql file is copied to EC2. It contains a create table command and insert commands to add data to the new table.
3. We need the epel repo (Extra Packages for Enterprise Linux) to download a MySQL client used in the ansible configuration.

```
resource "aws_instance" "example" {
  ami           = data.aws_ssm_parameter.webserver-ami.value
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name      = aws_key_pair.aws-key.key_name

  tags = {
    Name = "ansible_install_mariadb"
  }

  connection {
    type     = "ssh"
    user     = "ec2-user"
    private_key = file(var.ssh_key_private)
    host     = self.public_ip
  }

  # Copy the playbook from local machine to EC2
  provisioner "file" {
    source      = "${path.root}/install_mariadb.yaml"
    destination = "install_mariadb.yaml"
  }

  # Copy the webserver content from local machine to EC2
  provisioner "file" {
    source      = "${path.root}/quotes.sql"
    destination = "quotes.sql"
  }

  # Execute a script on a remote resource
  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y && sudo amazon-linux-extras install ansible2 -y",
      "sleep 60s",
      "sudo amazon-linux-extras install epel -y",
      "sudo yum-config-manager --enable epel",
      "ansible-playbook install_mariadb.yaml"
    ]
  }
}
```

Ansible Playbook – Database (4 of 6)

Reviewing the playbook:

1. Install Maria db
2. Start and enable Maria DB
3. Install My SQL client, which enables steps 4 and 5
4. Create a new database schema in addition to those created by default
5. Create a new user. The password is entered in plain text, we will learn a better approach in the next class.
6. Execute My SQL command to run the sql file against the new schema.

```
- name: Install mariadb
  hosts: localhost
  remote_user: ec2-user
  become: true

  tasks:
    - name: install mariadb
      yum:
        name: mariadb-server
        state: present
    - name: Start and Enable Mariadb
      systemd:
        name: mariadb
        state: started
        enabled: true
    - name: install MySQL client library to allow use of mysql_user
      yum:
        name: python2-PyMySQL
        state: present
    - name: Create a new database with name 'bobdata'
      mysql_db:
        name: bobdata
        state: present
    - name: create user
      mysql_user:
        name: Bob
        password: pa55word
        priv: '*.*:ALL'
    - name: shell to create table and import data
      ansible.builtin.shell:
        cmd: mysql -u root bobdata < quotes.sql
```

Ansible Playbook - Database (5 of 6)

Executing the terraform code, we can review the ansible output, which maps back to the 6 steps in the previous slide.

1. Install Maria db
2. Start and enable Maria DB
3. Install My SQL client
4. Create a new database schema
5. Create a new user.
6. Execute My SQL command to run the sql file against the new schema.

```
aws_instance.example (remote-exec): PLAY [Install mariadb] *****
aws_instance.example (remote-exec): TASK [Gathering Facts] *****
aws_instance.example: Still creating... [2m30s elapsed]
aws_instance.example (remote-exec): ok: [localhost]

aws_instance.example (remote-exec): TASK [install mariadb] *****
aws_instance.example: Still creating... [2m40s elapsed]
aws_instance.example: Still creating... [2m50s elapsed]
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [Start and Enable Mariadb] *****
aws_instance.example: Still creating... [3m0s elapsed]
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [install MySQL client library to allow use of mysql_user] *****
aws_instance.example: Still creating... [3m10s elapsed]
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [Create a new database with name 'bobdata'] *****
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [create user] *****
aws_instance.example (remote-exec): [WARNING]: Module did not set no_log for update_password
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [shell to create table and import data] *****
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): PLAY RECAP *****
aws_instance.example (remote-exec): localhost : ok=7 changed=6 unreachable=0

aws_instance.example: Creation complete after 3m19s [id=i-0d094ff99b9a5af94]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

Ansible-Install_httpd-PublicIP = "52.23.211.52"
```


Ansible Playbook – Database (6 of 6)

Next, let's log on to our database server and verify that everything ran as expected.

1. Log on to server.
2. Check My SQL version.
3. Logon as the user created through playbook
4. Use show databases command to see new bobdata schema
5. Select * from the new table created by playbook to verify that it was executed successfully.

You've now verified that the database install, user and table creation was successful.

shane@Shanes-MacBook-Pro 01.SimpleDBserver-Playbook % ssh ec2-user@52.23.211.52
Last login: Thu Jul 13 10:34:17 2023

```
--|  --|  )  
_| (  --| /  
---| \---|  
Amazon Linux 2 AMI
```

```
https://aws.amazon.com/amazon-linux-2/  
No packages needed for security; 2 packages available  
Run "sudo yum update" to apply all updates.  
[ec2-user@ip-172-31-94-123 ~]$ mysql --version  
mysql Ver 15.1 Distrib 5.5.68-MariaDB, for Linux (x86_64) using readline 5.1  
[ec2-user@ip-172-31-94-123 ~]$ mysql -u Bob -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ;  
Your MariaDB connection id is 7  
Server version: 5.5.68-MariaDB MariaDB Server
```

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type 'c' to clear the current input statement.

MariaDB [(none)]> show databases;

Database
information_schema
bobdata
mysql
performance_schema
test

5 rows in set (0.00 sec)

MariaDB [(none)]> select * from bobdata.SteveJobsQuotes;

id	quote
1	Stay hungry, stay foolish.
2	Innovation distinguishes between a leader and a follower.
3	Your work is going to fill a large part of your life, and the only way to be truly satisfied is to do what you believe is great work.
4	The people who are crazy enough to think they can change the world are the ones who do.
5	Remembering that you are going to die is the best way I know to avoid the trap of thinking you have something to lose. You are already naked. There is no reason not to follow your heart.
6	Innovation comes from people meeting up in the hallways or calling each other at 10:30 at night with a new idea or because they realized something that shoots holes in how we've been thinking about a problem.
7	The people who are doing the work are the moving force behind the Macintosh. My job is to create a space for them, to clear out the rest of the organization and keep it at bay.
8	Design is not just what it looks like and feels like. Design is how it works.
9	You have to trust in something – your gut, destiny, life, karma, whatever. This approach has never let me down, and it has made all the difference in my life.
10	Quality is more important than quantity. One home run is much better than two doubles.
11	My favorite things in life don't cost any money. It's really clear that the most precious resource we all have is time.
12	Innovation is the ability to see change as an opportunity – not a threat.
13	The only way to do great work is to love what you do.

Modules

Modules

- We have used some common modules as part of the web and database server creation. They can be used as part of ad hoc commands or as part of playbooks.
- Specialized module are preferred instead of the command or shell module, due to built in error handling and idempotency that these modules provide. *
- An operation is idempotent if the result of performing it once is the same as the result of performing it repeatedly without any intervening actions. **



Photo by [Vlado Paunovic](#) on [Unsplash](#)

* <https://spacelift.io/blog/ansible-best-practices#4-modules-best-practices>

** https://docs.ansible.com/ansible/latest/reference_appendices/glossary.html#

Modules

As you have seen through the playbooks we have created so far, modules are a powerful way of encapsulating behavior and providing consistent outcomes.

The top 5 in this table includes modules from our playbooks, along with 5 others that are also quite popular. *

Module Name	Purpose
yum	Popular repository download utility
systemd	Controls systemd units e.g. services, timers
unarchive	Unzip a file on remote server
copy	Copy files from control node to remote server
mysql_*	Modules for MySQL db management
lineinfile	Modify lines in existing files on remote server
file	Modify file state and permissions
user	Manage user accounts on remote server
command	Execute commands on remote server
git	Manage git checkouts for deployments

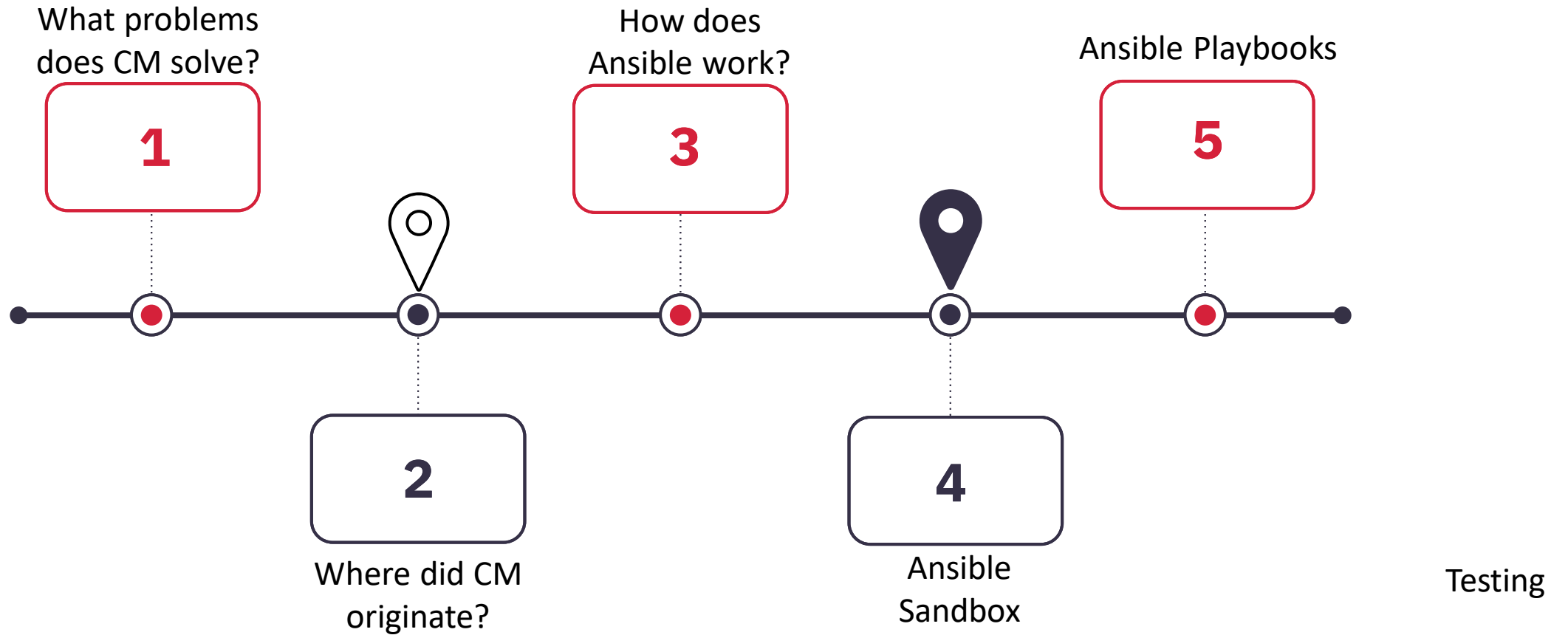
* <https://opensource.com/article/20/10/ansible-modules-linux>
<https://www.whizlabs.com/blog/ansible-modules/>

Custom Modules

- If you cannot find a module that meets your requirements, you can also create your own.
- Modules for inclusion in Ansible itself must be Python or Powershell.
- To create a module, you need ansible-core installed, so that your code can be verified.
- All modules return state in json format, so your custom module will need to.
- Unit testing and integration testing of your module are both important.
- Finally, documentation for your module is also important.



Journey Map



Wrap up

- Q & A
- What's 1 thing you learned?
- What's 1 challenge?

References

- Images [Slide 1] used under license from Microsoft PowerPoint stock images.
- Image [Slide 12] 2020 "Woman Sitting on Bench Looking at Mountains" by Yan Krukau used under free license from Pexels.